CSE638 GRS 2026

Performance Comparison of Kubernetes CNI Plugins

Flannel vs Calico vs Cilium

Group Number: 5
Ayush Jain - MT25066
Singh Tharun - MT25087
Ruchir Jain - MT25080

Contents

1 Problem Statement **3**

2 Why This Problem Matters **3**

3 Objectives **3**

4 Background (Short) **4**

4.1 Flannel 4

4.2 Calico 4

4.3 Cilium 4

5 Methodology **4**

5.1 What We Kept Constant 4

5.2 Metrics Collected 4

6 Installation and Run Steps **5**

7 Results and Observations **5**

7.1 p99 Latency Comparison 5

7.2 Latency Variability Across Runs 6

7.3 CPU and Memory Overhead 7

8 Why We Focused on p99 **7**

9 Sample Data from CSV Files **8**

9.1 Top 5 Rows from Latency CSVs (QPS 100) 8

9.2 Top 5 Rows from Resource CSVs 8

10 Conclusion **9**

11 References **9**

1 Problem Statement

In Kubernetes, pod networking depends on the CNI plugin. Each plugin works in its own way. Because of that, the same app can show different latency and different CPU/memory usage.

In our project, we are comparing all three popular CNI plugins like flannel , calico and cilium.

So we ran the same workload on all three and compared the output.

2 Why This Problem Matters

For production systems, networking affects:

User response time

Cost of running the cluster

Stability during high traffic

So the CNI choice is not just a setup step, it changes real performance. Our goal was to make this choice easier using actual measurements by analysing graph and all.

3 Objectives

Our goals were:

1. Build the same benchmark setup for all three CNIs.
2. Measure latency carefully, especially tail latency (p99).
3. Measure CPU and memory overhead.
4. Explain results in simple language.

4 Background (Short)

4.1 Flannel

Flannel is simple and lightweight. In many cases, it is easy to manage and uses fewer resources.

4.2 Calico

Calico is strong for policy and routing. It usually gives a good balance of features and performance and hence this can also be seen in our experiment.

4.3 Cilium

Cilium uses eBPF. It gives advanced networking features and better visibility.

5 Methodology

5.1 What We Kept Constant

To keep the comparison fair, we kept these things same in every run:

Machine and environment

Workload type

Deployment process

Measurement method

5.2 Metrics Collected

Latency: p50, p90, p99, average

Throughput and error rate

Resource usage: CPU (mcores) and Memory (MiB)

6 Installation and Run Steps

From repository root:

```
chmod +x G_5_scripts/*.sh
./G_5_scripts/G_5_setup_prereqs.sh
```

For each CNI (flannel/calico/cilium):

```
./G_5_scripts/G_5_create_cluster.sh <cni>
./G_5_scripts/G_5_deploy_workload.sh <cni>
./G_5_scripts/G_5_run_benchmark.sh <cni>
./G_5_scripts/G_5_collect_metrics.sh <cni>
```

Plot generation:

```
./G_5_artifacts/G_5_venv/bin/python G_5_plots/G_5_plot_latency_hardcoded.py
./G_5_artifacts/G_5_venv/bin/python G_5_plots/G_5_plot_latency_variability_hardcoded.
./G_5_artifacts/G_5_venv/bin/python G_5_plots/G_5_plot_overhead_hardcoded.py
```

7 Results and Observations

7.1 p99 Latency Comparison

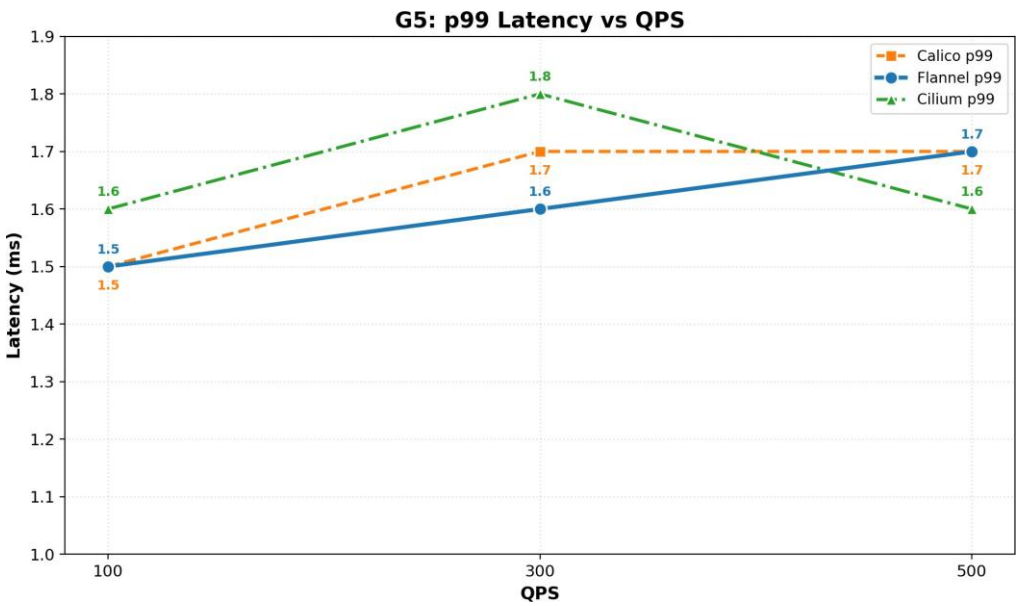

Figure 1: p99 latency comparison for Flannel, Calico, and Cilium.

What we saw: All three CNIs were close in latency. The differences were small in our test range.

7.2 Latency Variability Across Runs

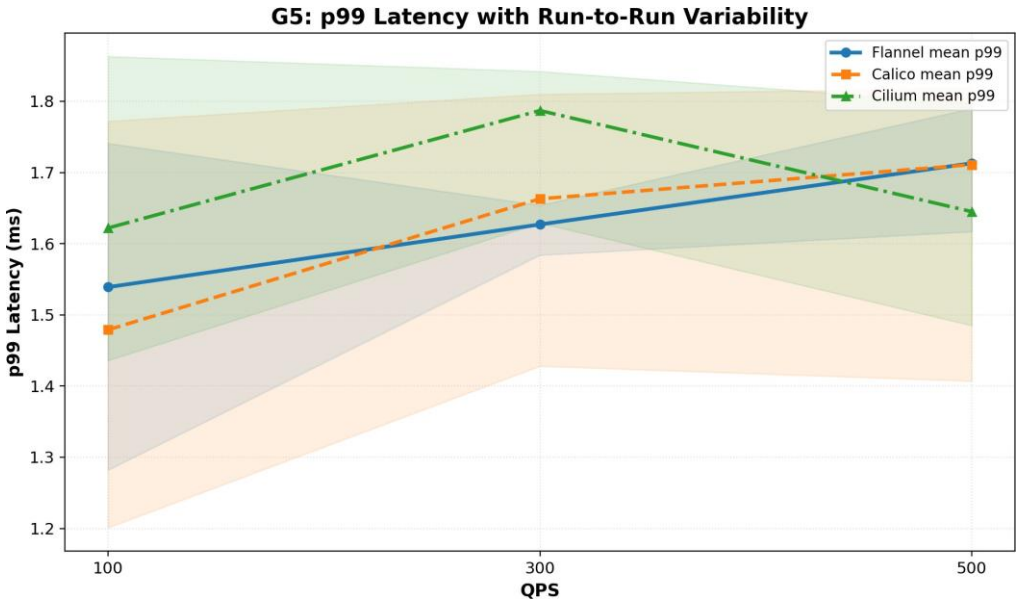

Figure 2: Mean p99 latency with min-max spread across repeated runs.

7.3 CPU and Memory Overhead

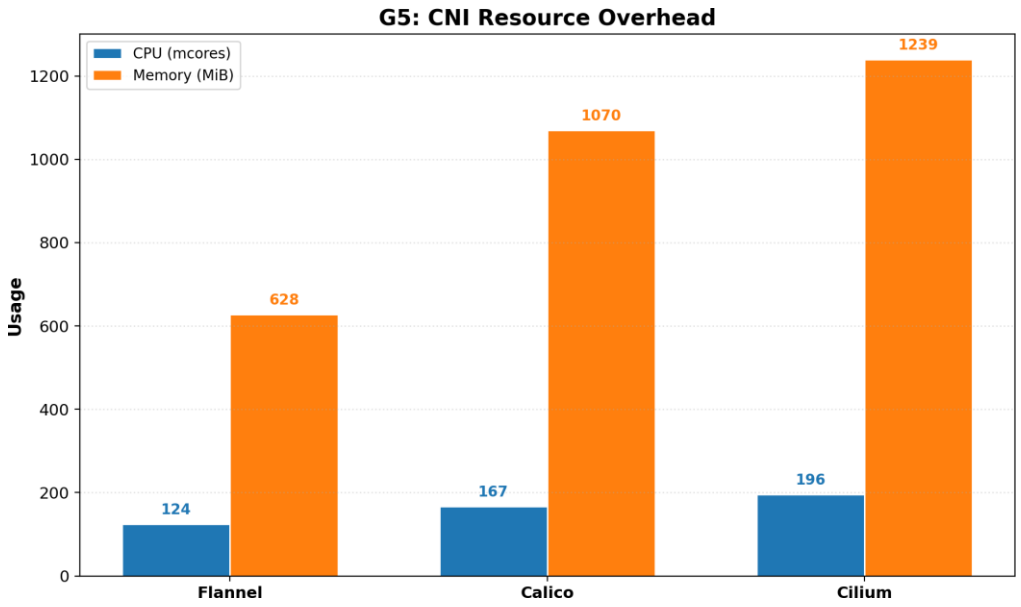

Figure 3: Aggregate CPU and memory overhead.

8 Why We Focused on p99

Average latency can look good even when some requests are very slow. In real apps, users notice these slow requests.

p99 helps us capture that slow tail clearly.

So we kept p50, p90, and average for context, but used p99 as the main reliability indicator.

9 Sample Data from CSV Files

9.1 Top 5 Rows from Latency CSVs (QPS 100)

Table 1: Flannel latency sample rows.

Run	p50	p90	p99	avg	throughput
1	0.00054596	0.00092766	0.00159493	0.00059488	99.9946
2	0.00056824	0.00094095	0.00174056	0.00062646	99.9945
3	0.00052654	0.00092067	0.00128243	0.00057646	99.9954
4	0.00055438	0.00093574	0.00158489	0.00059925	99.9948
5	0.00056222	0.00093156	0.00149200	0.00059302	99.9937

Table 2: Calico latency sample rows.

Run	p50	p90	p99	avg	throughput
1	0.00055235	0.00094075	0.00174991	0.00060487	99.9935
2	0.00055305	0.00094226	0.00136714	0.00059748	99.9923
3	0.00055499	0.00094422	0.00177175	0.00061223	99.9945
4	0.00055118	0.00092480	0.00120133	0.00055425	99.9953
5	0.00055501	0.00092518	0.00130327	0.00056306	99.9933

Table 3: Cilium latency sample rows.

Run	p50	p90	p99	avg	throughput
1	0.00055781	0.00094486	0.00179226	0.00066001	99.9953
2	0.00057691	0.00096374	0.00186320	0.00068801	99.9927
3	0.00056278	0.00095677	0.00145410	0.00067041	99.9944
4	0.00057342	0.00094929	0.00143620	0.00064682	99.9956
5	0.00055447	0.00095056	0.00156517	0.00064484	99.9932

9.2 Top 5 Rows from Resource CSVs

Table 4: Flannel resource sample rows.

Namespace	Pod	CPU (mcores)	Memory (MiB)
g5-bench	g5-client-d57c4d46-6q6rl	1	5
g5-bench	g5-echo-649fbccd57-7t4wj	0	10
g5-bench	g5-echo-649fbccd57-d7dmb	0	10
g5-bench	g5-echo-649fbccd57-dj2fn	0	10
kube-flannel	kube-flannel-ds-nfcnd	9	14

Table 5: Calico resource sample rows.

Namespace	Pod	CPU (mcores)	Memory (MiB)
g5-bench	g5-client-d57c4d46-n6j2q	0	4
g5-bench	g5-echo-649fbccd57-ck6rc	0	11
g5-bench	g5-echo-649fbccd57-pztgs	0	10
g5-bench	g5-echo-649fbccd57-xxsnk	0	10
kube-system	calico-kube-controllers-5b4bd99c9c-ln7j6	4	13

Table 6: Cilium resource sample rows.

Namespace	Pod	CPU (mcores)	Memory (MiB)
g5-bench	g5-client-d57c4d46-9kwz7	1	4
g5-bench	g5-echo-649fbccd57-42f7m	0	8
g5-bench	g5-echo-649fbccd57-8jvjg	0	9
g5-bench	g5-echo-649fbccd57-qrlc5	0	8
kube-system	cilium-envoy-7j8pv	11	23

10 Conclusion

This comparison gave us a clear and practical picture:

All three CNIs gave solid latency performance in our tested load range.

The biggest difference was resource overhead.

Flannel was most lightweight in this setup.

Calico gave a balanced profile.

Cilium offered strong features with higher resource cost.

11 References

1. https://kubernetes.io/docs/
2. https://github.com/flannel-io/flannel

3. https://docs.tigera.io/calico/latest

4. https://docs.cilium.io/